

Zero Knowledge proofs

Grupe

Grupa je skup G zajedno sa operacijom $*$, tako da važe sledeća svojstva:

1. **Zatvorenost:** ako su a i b elementi skupa G , onda je i $a * b$ element skupa G
2. **Asocijativnost:** za sve $a, b, c \in G$ važi $(a * b) * c = a * (b * c)$
3. **Neutralni element:** postoji element $e \in G$ takav da za svako $a \in G$ važi $e * a = a * e = a$
4. **Inverzni element:** za svako $a \in G$ postoji element $a^{-1} \in G$ takav da važi: $a * a^{-1} = a^{-1} * a = e$

Primeri grupa

- **Primer 1:** $(\mathbb{N}_0, +)$ nije grupa
 - Zatvorenost: DA
 - Asocijativnost: DA
 - Neutralni element: DA, to je 0
 - Inverzni element: NE
- **Primer 2:** $(\mathbb{Z}, +)$ je grupa
 - Zatvorenost: DA
 - Asocijativnost: DA
 - Neutralni element: DA, to je 0
 - Inverzni element: DA, za a je inverz $-a$
- **Primer 3:** $(\mathbb{Z}, \cdot)$ nije grupa
 - Zatvorenost: DA
 - Asocijativnost: DA
 - Neutralni element: DA, to je 1
 - Inverzni element: NE
- **Primer 4:** $(\mathbb{Q} \setminus \{0\}, \cdot)$ je grupa
 - Zatvorenost: DA
 - Asocijativnost: DA
 - Neutralni element: DA, to je 1
 - Inverzni element: DA, za a je inverz $1/a$

Ciklična grupa $\mathbb{Z}_p^*$

Neka je p prost broj. $\mathbb{Z}_p^*$ je skup $\{1, 2, \dots, p-1\}$. Operacija u ovoj grupi je množenje po modulu p , definisana sa:

$$a \cdot b = (a * b) \pmod{p}$$

Ova struktura je grupa jer važi:

- rezultat množenja opet pripada Z_p^*
- operacija je asocijativna
- neutralni element je 1
- svaki element ima inverz po modulu p

Primer: $Z_{11}^* = \{1, 2, 3, \dots, 10\}$

Posmatrajmo stepene broja 2 po modulu 11:

2, 4, 8, 5, 10, 9, 7, 3, 6, 1

Dobili smo sve elemente iz Z_{11}^* . Zato je 2 **generator** grupe Z_{11}^* .

Generator grupe

Neka je G grupa.

Element $g \in G$ nazivamo **generatorom** grupe G ako se pomoću njegovih stepena mogu dobiti svi elementi grupe.

Ako grupa ima bar jedan generator, kažemo da je grupa **ciklična**.

Za prost broj p , grupa Z_p^* je uvek ciklična.

Da li postoji brži način da proverimo da li je neki element generator?

Provera generatora u Z_p^*

Neka je p prost broj i neka je g element grupe Z_p^* .

Element g je generator grupe Z_p^* ako za svaki prost delilac q broja $p - 1$ važi:

$$g^{\frac{p-1}{q}} \neq 1 \pmod{p}$$

Primer: Z_{11}^*

$$p - 1 = 10 = 2 * 5$$

Prosti delioci broja 10 su 2 i 5.

- Proveravamo da li je 2 generator:

$$2^5 \pmod{11} = 10 \neq 1$$

$$2^2 \pmod{11} = 4 \neq 1$$

Zato je 2 generator grupe Z_{11}^* .

- Proveravamo da li je 3 generator:

$$3^5 \pmod{11} = 1$$

Zato 3 nije generator grupe Z_{11}^* .

Generatori grupe Z_{11}^* su: 2, 6, 7 i 8.

Ciklična grupa Z_n^*

Opštije, postoji i Z_n^* , gde n ne mora biti prost broj. Tada u grupu ne ulaze svi nenulti elementi, već samo oni koji su uzajamno prosti sa n .

Ta grupa se najviše koristi u RSA kriptografiji.

Problem diskretnog logaritma (Discrete logarithm problem)

Dato je:

- p – prost broj
- g – generator grupe Z_p^*
- $b \in Z_p^*$

Treba pronaći x takvo da:

$$g^x \equiv b \pmod{p}$$

odnosno:

$$x = \log_g b$$

Primer: Element $g = 7$ je generator grupe Z_{17}^* . Odrediti $\log_7 8$ u grupi $Z_{17}^* = \{1, 2, \dots, 16\}$

Računamo stepene broja 7 po modulu 17 i dobijamo:

$$7^{14} \pmod{17} = 8$$

$$\text{Dakle: } \log_7 8 = 14$$

Problem je lak za proveru, ali težak za rešavanje kada su brojevi veliki.

Ako znamo x , lako računamo $g^x \pmod{p}$.

Ako znamo samo g, p i b , teško je pronaći x .

Konačno polje F_p

Polje je skup u kome možemo da sabiramo, oduzimamo, množimo i delimo nenultim elementima.

U kriptografiji koristimo konačno polje:

$$F_p = \{0, 1, 2, \dots, p-1\}, \text{ gde je } p \text{ prost broj.}$$

Sve operacije u F_p računaju se po modulu p .

Važno je razlikovati F_p kao polje i $F_p^* = F_p \setminus \{0\}$ kao multiplikativnu grupu.

Elipsičke krive

Elipsička kriva je skup tačaka koje zadovoljavaju jednačinu oblika:

$$E : y^2 = x^3 + ax + b$$

gde su a i b elementi nekog polja.

Najčešće posmatramo elipsičke krive nad konačnim poljem $F_p = \{0, 1, 2, \dots, p-1\}$, gde je p veliki prost broj. Sabiranje i množenje se rade po modulu p .

Skup tačaka na elipsičkoj krivi, zajedno sa posebnom tačkom O , formira grupu. Tačka O se naziva tačkom u beskonačnosti i ima ulogu neutralnog elementa.

Primer:

Posmatrajmo elipsičku krivu nad poljem F_{11} :

$$E/F_{11} : y^2 = x^3 + 4x + 3$$

Njene tačke su parovi (x, y) iz F_{11} koji zadovoljavaju jednačinu krive:

$(0, 5), (0, 6), (3, 3), (3, 8), (5, 4), (5, 7), (6, 1), (6, 10), (7, 0), (9, 3), (9, 8), (10, 3), (10, 8), O$ (tačka u beskonačnosti).

Ukupno ima 14 tačaka.

Sabiranje tačaka

Ako imamo dve tačke P i Q :

1. povučemo pravu kroz P i Q
2. ta prava seče krivu u još jednoj tački
3. tu tačku preslikamo preko x -ose
4. dobijena tačka R predstavlja zbir tačaka P i Q .

Ako su P i Q iste tačke, koristi se tangenta u tački P .

Ovako definisano sabiranje tačaka ćemo najčešće označavati sa $\oplus$.

Elipsička kriva kao grupa

Skup tačaka na elipsičkoj krivi, zajedno sa operacijom sabiranja $\oplus$, formira Abelovu grupu, odnosno važe zatvorenost, asocijativnost, komutativnost i postoje neutralni element i inverzni element.

- Neutralni element je tačka O , odnosno tačka u beskonačnosti.
- Ako je $P = (x, y)$, onda je njen inverz tačka $-P = (x, -y)$.

Primer 1: Elipsička kriva nad poljem realnih brojeva

$$E/\mathbb{R} : y^2 = x^3 - 2x$$

Primetimo da tačke $P(-1, -1)$ i $Q(0, 0)$ pripadaju krivoj E . Hoćemo da nađemo R , tako da je $R = P \oplus Q$.

Prava koja sadrži tačke P i Q ima jednačinu: $y = x$.

Presek te prave i elipsičke krive je tačka $(2, 2)$.

Dakle, $P \oplus Q = R(2, -2)$.

Primer 2: Elipsička kriva nad poljem F_{11}

$$E/F_{11} : y^2 = x^3 - 2x$$

Primetimo da tačke $P(5, 7)$ i $Q(8, 10)$ pripadaju krivoj E/F_{11} . Hoćemo da nađemo tačku $R = P \oplus Q$.

Prava koja prolazi kroz tačke P i Q ima jednačinu $y = x + 2$.

Presek te prave i krive je tačka $(10, 1)$, pa je R tačka sa koordinatama $(10, -1)$, odnosno $(10, 10)$.

Formule za sabiranje na elipsičkoj krivi

Formula za sabiranje različitih tačaka

Sabiramo tačke $P(x_1, y_1)$ i $Q(x_2, y_2)$, gde je $x_1 \neq x_2$, na elipsičkoj krivoj $y^2 = x^3 + ax + b$.

Koeficijent pravca prave kroz tačke P i Q je $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$.

Izjednačavanjem jednačine prave i elipsičke krive dobijamo x -koordinatu treće presečne tačke:

$$x_3 = \lambda^2 - x_1 - x_2$$

Zatim dobijamo i y -koordinatu:

$$y_3 = \lambda(x_3 - x_1) + y_1$$

Rezultujuću tačku dobijamo simetrijom u odnosu na x -osu:

$$R = P \oplus Q = (x_3, -y_3)$$

Formula za dupliranje tačke

Dupliramo tačku $P(x_1, y_1)$ na eliptičkoj krivoj $y^2 = x^3 + ax + b$.

Koeficijent pravca tangente na krivu u tački P je $\lambda = \frac{3x_1^2 + a}{2y_1}$.

Izjednačavanjem jednačine tangente i eliptičke krive dobijamo x -koordinatu presečne tačke:

$$x_3 = \lambda^2 - 2x_1$$

Zatim dobijamo i y -koordinatu rezultujuće tačke:

$$y_3 = \lambda(x_3 - x_1) + y_1$$

Rezultujuću tačku dobijamo simetrijom u odnosu na x -osu:

$$R = P \oplus P = (x_3, -y_3)$$

Primer: Eliptička kriva nad poljem F_{23}

$$E/F_{23} : y^2 = x^3 + 5x + 7$$

Primetimo da tačka $P(2, 5)$ pripada krivoj E/F_{23} . Pronaći R , tako da $R = P \oplus P$.

$$y(x) = \sqrt{x^3 + 5x + 7} \implies y'(x) = \frac{1}{2} \cdot \frac{3x^2 + 5}{\sqrt{x^3 + 5x + 7}}$$

$$k = y'(2) = 17 \cdot 10^{-1} = 17 \cdot 7 = 119 \equiv 4 \pmod{23}$$

$$l : y = 4x + 20 \implies x_R = 12$$

Dobijamo tačku $R(12, 1)$.

Množenje skalarom

Double-and-add algoritam

Tačku P možemo da pomnožimo skalarom m tako što izvršimo $m - 1$ sabiranja:

$$m * P = P \oplus P \oplus P \oplus P \oplus \dots \oplus P$$

Ali ovaj metod je previše spor za veliko m !

Za brže računanje broj m zapisujemo u binarnom obliku i rezultat dobijamo u logaritamskom vremenu.

Na primer, da bismo izračunali $79 * P$, prvo broj 79 pretvaramo u binarni zapis. Zatim možemo izračunati zbir:

$$79 * P = 2^6 * P \oplus 2^3 * P \oplus 2^2 * P \oplus 2^1 * P \oplus 2^0 * P$$

Multi-Scalar Multiplication

Računski najzahtevniji deo u algoritmu za generisanje dokaza kod većine ZK sistema zasnovanih na eliptičkim krivama jeste algoritam za množenje tačaka skalarom, odnosno Multi-Scalar Multiplication (MSM).

- Naivni algoritam koristi double-and-add algoritam.
- Najbrži pristup je varijanta Pippenger-ovog algoritma, koja se naziva bucket method.

Kriptografija zasnovana na eliptičkim krivama (ECC)

Problem diskretnog logaritma na eliptičkoj krivoj

Neka je P tačka na eliptičkoj krivoj i neka je $Q = m * P$.

Ako su nam poznate tačke P i Q , problem pronalaženja broja m naziva se problem diskretnog logaritma na eliptičkim krivama, odnosno **Elliptic Curve Discrete Logarithm Problem (ECDLP)**.

Dakle, lako je izračunati $Q = m * P$, ali je veoma teško iz poznatih P i Q odrediti m .

Sigurnost

Grupe eliptičkih krivih relativno male veličine pružaju isti nivo sigurnosti kao multiplikativne grupe nad mnogo većim konačnim poljima.

Na primer, eliptička kriva nad poljem veličine 160 bita daje nivo sigurnosti uporediv sa klasičnim sistemom zasnovanim na diskretnom logaritmu u konačnom polju veličine 1248 bita.

Privatni (private) i javni (public) ključ

Neka je G generator tačka na eliptičkoj krivoj nad konačnim poljem F_p , gde je p prost broj.

Neka su $prA, prB \in F_p$ privatni ključevi dve različite osobe, Alise i Bobana.

Tada su:

- $pubA = prA * G$
 - $pubB = prB * G$
- njihovi javni ključevi.

Diffie-Hellman razmena ključa

Cilj je da dve osobe naprave zajednički tajni ključ preko javnog kanala.

- Alisa bira privatni ključ a i objavljuje svoj javni ključ $A = a * G$.
- Boban bira privatni ključ b i objavljuje svoj javni ključ $B = b * G$.

Alisa i Boban mogu da izračunaju zajednički tajni ključ:

- Alisa računa $a * B = a * (b * G) = (ab) * G$
- Boban računa $b * A = b * (a * G) = (ab) * G$

Napadač vidi G, A i B , ali zbog problema diskretnog logaritma ne može lako da izračuna a i b , a samim tim ni Bobanov i Alisin zajednički tajni ključ.

Uparivanje na eliptičkoj krivoj (Elliptic curve pairing)

Definicija:

Neka je E eliptička kriva nad konačnim poljem K . Neka su $\mathbb{G}_1$ i $\mathbb{G}_2$ aditivno zapisane podgrupe reda p , gde je p prost broj, eliptičke krive E , i neka su $g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2$ generatori grupa $\mathbb{G}_1$ i $\mathbb{G}_2$ redom. Preslikavanje $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, gde je $\mathbb{G}_T$ multiplikativno zapisana podgrupa od K reda p , naziva se uparivanje na eliptičkoj krivi (elliptic curve pairing) ako zadovoljava sledeće uslove:

1. $e(g_1, g_2) \neq 1$
2. $\forall R, S \in \mathbb{G}_1, \forall T \in \mathbb{G}_2 : e(R + S, T) = e(R, T) * e(S, T)$
3. $\forall R \in \mathbb{G}_1, \forall S, T \in \mathbb{G}_2 : e(R, S + T) = e(R, S) * e(R, T)$

Sledeća svojstva uparivanja na eliptičkoj krivi se mogu lako proveriti:

1. $\forall S \in \mathbb{G}_1, \forall T \in \mathbb{G}_2 : e(S, -T) = e(-S, T) = e(S, T)^{-1}$
2. $\forall S \in \mathbb{G}_1, \forall T \in \mathbb{G}_2 : e(a * S, b * T) = e(b * S, a * T) = e(S, T)^{a*b}$

Tipovi uparivanja:

- **Type 1:** $\mathbb{G}_1 = \mathbb{G}_2$, i kažemo da je e simetrično bilinearno preslikavanje;
- **Type 2:** $\mathbb{G}_1 \neq \mathbb{G}_2$ i postoji efikasan homomorfizam $\phi : \mathbb{G}_2 \rightarrow \mathbb{G}_1$, ali ne postoji efikasan u drugom smeru;
- **Type 3:** $\mathbb{G}_1 \neq \mathbb{G}_2$ i ne postoji efikasan homomorfizam između $\mathbb{G}_1$ i $\mathbb{G}_2$.

Post-kvantna kriptografija

Današnja asimetrična kriptografija se oslanja na sledeće probleme:

- RSA: faktORIZACIJA velikih brojeva
- Diffie-Hellman: diskretni logaritam u Z_p^*
- ECC: diskretni logaritam na eliptičkim krivama

Dovoljno snažan kvantni računar mogao bi efikasno da reši ove probleme pomoću Šorovog (Peter Shor) algoritma.

Zato se uvodi post-kvantna kriptografija: kriptografija koja se izvršava na običnim računarima, ali je dizajnirana da bude otporna i na kvantne računare.

Simetrična kriptografija nije razbijena na isti način. AES i hash funkcije ostaju upotrebljivi, ali se obično preporučuju veći sigurnosni parametri.

Godine 2017. Američki nacionalni institut za standarde i tehnologiju (NIST) pokrenuo je proces standardizacije novih kriptografskih algoritama koji bi bili otporni na napade kvantnih računara. Nakon nekoliko krugova evaluacije, NIST je izabrao sledeće post-kvantne algoritme:

Key Encapsulation Mechanisms (KEM):

- **CRYSTALS-Kyber (ML-KEM)**

Module-Lattice-based Key Encapsulation Mechanism

→ Primarna post-kvantna zamena za mehanizme razmene ključa zasnovane na RSA, Diffie-Hellmanu i eliptičkim krivama.

- **HQC (Hamming Quasi-Cyclic)**

Code-based Key Encapsulation Mechanism

→ Rezervni KEM zasnovan na kodovima, izabran kao dodatna alternativa za standardizaciju.

Algoritmi za digitalno potpisivanje:

- **CRYSTALS-Dilithium (ML-DSA)** (Module-Lattice-based Digital Signature Algorithm)
- **FALCON** (Lattice-based Digital Signature Algorithm (NTRU lattice))
- **SPHINCS+ (SLH-DSA)** (Stateless Hash-based Digital Signature Algorithm)

Lattice-based kriptografija

Shortest Vector Problem (SVP) i Closest Vector Problem (CVP)

Lattice, odnosno rešetka, je skup tačaka koje dobijamo celobrojnim kombinovanjem baznih vektora. Mnogi post-quantni algoritmi koriste činjenicu da su određeni problemi na rešetkama, veoma teški za rešavanje. Dva najosnovnija problema su:

- **SVP (Shortest Vector Problem):** među svim tačkama rešetke koje nisu 0, treba pronaći onu koja je najbliža koordinatnom početku.
- **CVP (Closest Vector Problem):** data je rešetka i jedna tačka koja ne mora da pripada rešetki. Cilj je pronaći tačku rešetke koja je najbliža toj zadatoj tački.

Zašto su ovi problemi teški?

SVP i CVP su laki za razumevanje, ali veoma teški za rešavanje u velikom broju dimenzija.

U kriptografiji se koriste lattice-i sa stotinama ili hiljadama dimenzija. Broj mogućih kandidata tada raste eksponencijalnom brzinom.

Pored toga, baza lattice-a može biti veoma iskrivljena, pa najkraći vektor ili najbliža tačka nisu očigledni. Zato su lattice problemi dobra osnova za post-quantnu kriptografiju.

Lattice-based problemi u praksi

U praksi se ne koriste uvek direktno baš SVP i CVP, već njihove efikasnije i praktičnije varijante.

Najpoznatije porodice problema su:

- **LWE (Learning With Errors);**
- **Ring-LWE;**
- **Module-LWE;**
- **SIS (Short Integer Solution);**
- **Module-SIS;**
- **NTRU problemi.**

Ove varijante uvode dodatnu strukturu i optimizacije, kako bi algoritmi bili brži i pogodniji za implementaciju.

FHE (Fully Homomorphic Encryption)

Prvu potpuno homomorfnu šemu šifrovanja predstavio je Craig Gentry 2009. godine u svojoj doktorskoj disertaciji.

Od tada je razvijen veliki broj FHE šema, uz značajan napredak u efikasnosti, bootstrapping tehnikama i praktičnoj upotrebljivosti.

Važno je da se moderne FHE šeme zasnivaju na lattice-based problemima, odnosno na Module-LWE.

Danas razvoj FHE-a guraju i akademska zajednica i industrija, sa sve većim fokusom na praktične primene nad šifrovanim podacima.

ZKP (Zero Knowledge Proof)

ZKP sistemi kombinuju tri glavne komponente:

- 1. aritmetizaciju problema
- 2. commitment šemu
- 3. protokol za proveru tvrdnje nad komitovanim podacima

U zavisnosti od toga koje se komponente koriste, dobijamo različite porodice ZK sistema. Dve najpoznatije porodice su:

- **SNARK** (Succinct Non-interactive Argument of Knowledge)
- **STARK** (Scalable Transparent Argument of Knowledge)

SNARK vs STARK

SNARK (Succinct Non-interactive Argument of Knowledge)	STARK (Scalable Transparent Argument of Knowledge)
• kratki dokazi • brza verifikacija • zahteva trusted setup (postoje i novije varijante bez trusted setup-a) • u klasičnim varijantama koristi eliptičke krive i pairings (postoje i post-kvantne varijante bez eliptičkih krivih)	• veći dokazi • transparentan setup • oslanja se na hash funkcije • otporan je na kvantne računare
Primeri: Groth16 PLONK sa KZG commitmentima	

Uvod u Circom

Šta je ZKP (Zero-Knowledge Proof)?

Zamisli da želiš da dokažeš prijatelju da si pronašao izlaz iz komplikovanog lavirinta na papiru, ali **ne želiš da mu pokažeš taj izlaz** kako mu ne bi pokvario rešavanje.

Zero-Knowledge Proof (Dokaz sa nultim znanjem) ti omogućava upravo to: **Da matematički dokažeš da znaš neku tajnu ili da si ispravno izračunao nešto, bez otkrivanja same tajne.**

U ovom protokolu uvek postoje dve uloge:

- 1. **Prover (onaj koji dokazuje):** Želi da dokaže da nešto zna (npr. ti koji znaš put kroz lavirint).

2. **Verifier (onaj koji proverava):** Proverava tvoj dokaz i odgovara samo sa **True** (dokaz je tačan) ili **False** (dokaz je netačan/lažan).

Šta je Circom?

Circom je programski jezik, ali nije kao Python, Java ili C. On se zove **HDL** (*Hardware Description Language*). Kada pisemo kod u Circomu, mi zapravo ne pišemo "algoritam" koji ide liniju po liniju, već **dizajniramo aritmetičko kolo** (zamisli elektronsko kolo sa žicama i logičkim kapijama, samo što ovde teku brojevi, a ne struja).

Sva matematika u ZKP svetu (pa i u Circomu) ne radi sa beskonačnim brojevima, već u **konačnom polju**. U prezentaciji se spominje ogroman broj `p = 21888242871839275...`. To znači da šta god računali, kolo operiše po modulu tog broja. Zato klasično programiranje ovde ne radi.

Osnove jezika Circom

U Circomu postoje tri osnovna pojma: **Signali, Ograničenja i Promenljive**.

A) Signali (`signal`)

Signali su "žice" u našem kolu. Oni nose vrednosti. Glavno pravilo za signale: **Jednom kada signalu dodeliš vrednost, ona više ne može da se menja.**

Signali se dele na:

- `signal input x;` – Ulazni podaci (mogu biti *private* ili *public*).
- `signal output y;` – Izlazni podaci kola.
- Pomoćni signali – Unutrašnje žice u kolu.

B) Ograničenja (`Constraints` `===`)

Ovo je srz ZKP-a. Ograničenja su matematička pravila koja tvoje kolo postavlja. Zapisuju se sa tri znaka jednako (`===`).

Ako ti u kodu napišeš `a * b === c;`, ti time kažeš: "Ovaj dokaz je validan AKO I SAMO AKO je A puta B zaista jednako C". Ako to nije slučaj, dokaz pada.

GLAVNO PRAVILO OGRANIČENJA: Sva ograničenja moraju biti maksimalno **kvadratna**. Šta to znači? Možemo pomnožiti maksimalno dva signala.

-  `a === b + c;` (linearno)
-  `a === b * c + d;` (kvadratno - okej je)
-  `a === b * c * d;` (NE MOŽE! Množiš tri signala).

Kako rešavamo problem množenja više od 2 broja (Primer sa repozitorijuma `multiplier3.circom`)?

Moramo "razbiti" izraz na više manjih uvođenjem pomoćnog signala:

```
signal input a;
signal input b;
signal input c;
signal output d;
```

```
signal tmp; // Pomoćna žica

tmp <== a * b;    // Prvo pomnožimo a i b
d <== tmp * c;    // Zatim pomnožimo rezultat (tmp) sa c
```

C) Operatori dodele (Razlika između `<--`, `==` i `<==`)

Ovo ljude najviše buni, a zapravo je vrlo logično:

- `<--` (Samo dodela): Izračunaj vrednost i stavi je u signal. Ovo **NE PRAVI OGRANIČENJE!**
- `==` (Samo ograničenje): Proverava pravilo, ne menja vrednosti.
- `<==` (DODELA I OGRANIČENJE): Najčešće se koristi. Istovremeno kaže "Stavi da je $y = x + 2$ " i stavi ograničenje "Zauvek proveriti da li je y jednako $x + 2$ ".

D) Promenljive (`var`)

Za razliku od signala, vrednost `var` (npr. `var x = 5;`) može da se menja. Promenljive se koriste samo kao pomoć **programeru dok pišeš kod** (npr. kao brojač u `for` petlji), one ne ulaze u samo kriptografsko kolo.

Problem "If/Else" uslova i Petlji

Pošto se iz tvog koda generiše statično kriptografsko kolo (zamrznut oblik), **veličina i struktura kola moraju biti poznati pre pokretanja**.

Zato **ne možeš** da napišeš `if (tajni_signal == 1) { uradi ovo } else { uradi ono }`.

Zašto? Zato što bi to menjalo strukturu kola u zavisnosti od korisnikovog unosa, a kolo mora biti statično!

Rešenje? Multiplekser (Mux)

Umesto klasičnog `if/else`, obično se u kolo ubacuju "oba" puta, a onda se pomoću formule odabere pravi. Biblioteka `circomlib` ima gotove Mux šablone za ovo.

Zadaci

Primer 1: Pogadanje broja (`pogodibroj.circom`)

- **Problem:** Igramo igru. Ja zamislim broj X . Ti probaš da ga pogodiš sa pokušajem Y . Želim da napravim program koji ti kaže "Tačno" ili "Netačno", a da **nikada nikome javno ne otkrijem moj broj X** ako nisi pogodio.
- **Problem naivne verzije:** Ako ja samo proveravam tvoj broj, ja bih mogao usred igre da varam i promenim moj broj X pre nego što ti odgovorim.
- **Rešenje: Commitment (Obavezivanje).**

Ja pre tvog pokušaja javno objavim tzv. "Commitment" C .

Kalkulacija je: $C = \text{Hash}(X, \text{salt})$.

`Salt` je samo neki veliki nasumičan broj (npr. `81471...`). Zašto dodajem `salt`? Zato što, ako bih heširao samo moj zamišljeni broj 5, ti bi mogao na svom kompjuteru da napraviš heševe svih brojeva od 1 do 100, uporediš ih sa mojim hešom i odmah otkriješ moj broj. Zbog nasumičnog `salt-a`, to ti je onemogućeno.

- **Šta kolo radi?** Kolo uzima moj **tajni** broj i **tajni** `salt`, kao i tvoj **javni** pokušaj i **javni** Commitment C .

Ono interno proveriti dve stvari:

1. Da li je $\text{Hash}(\text{moj_broj}, \text{moj_salt}) == C$ (da dokažem da nisam varao i menjao broj).

2. Da li je *moj_broj* === *tvoj_pokusaj*.

Ako sve prođe - sistem nam daje dokaz. Igra je rešena pametnim ugovorom!

Primer 2: Membership Proof - Merkle Stablo (`membershipproof.circom`)

- **Ideja:** Merkle stablo služi da se 10.000 korisnika kompresuje u jedan jedini "korenski heš" (Root Hash). Želiš da dokažeš da imaš pravo pristupa aplikaciji, bez otkrivanja tvog imena i naloga.
- **Kako radi?** Svi u sistemu znaju javni `Root Hash` baze podataka. Kolo prihvata tvoje lične podatke, tvoji lični heš, kao i tvoju putanju u stablu. Kolo tada ponovo izračuna putanju nagore. Ako se na samom vrhu dobijeni heš poklopi sa javnim `Root Hash`-om (`dobijeni_koren === javni_koren`), to apsolutno dokazuje da se ti zaista nalaziš u tom drvetu. Anonimnost sačuvana.

Primer 3: Range Proof (`rangeproof.circom`)

- **Ideja:** Hoćeš da dokažeš sajtu da si punoletan (imaš preko 18 godina), ali odbijaš da im kažeš da zapravo imaš 25 godina.
- **Kako radi?** Želimo da dokažemo da je naš tajni broj X unutar intervala $[A, B]$ (npr. između 18 i 100). Circom koristi ugrađene Templejte (šablone) iz `circomlib` biblioteke, konkretno komponente za upoređivanje (`LessThan`, `GreaterEqThan`). Kolo uzme tvoje godine i granice, komponenta vrati `1` ako je istina ili `0` ako je laž. Na kraju dodamo ograničenje: `rezultat_provere === 1`. Nema laganja, nema otkrivanja godina.

Primer 4: Privatni u Javni Ključ (`privKeyCheck.circom`)

- **Ideja:** Da bi poslao kriptovalute, ti koristiš svoj Privatni ključ da generišes Javni ključ. Želimo da ZKP-om dokažeš: "Znam koji je privatni ključ ovog novčanika", ali da ga logično ne prikažeš u kolu, inače bi ostao bez para.
- **Kako radi?** Koristi se napredna matematika po imenu Eliptičke krive (EdDSA i `escalarmulfix` iz `circomlib-a`). Kolo naprosto unutra pokrene kompleksno kriptografsko množenje nad tvojim unetim privatnim ključem. Zatim uporedi rezultat sa javnim ključem: `dobijeni_javni_kljuc === uneti_javni_kljuc`.

Korak po Korak - Životni ciklus jednog kola

Ovaj deo iz prezentacije je jako bitan za razumevanje šire slike. Kako od Circom koda dolazimo do ZKP dokaza?

1. **Pišemo kolo (Circom):** Gde deklariramo šta je javno, šta je privatno.
2. **Kompajliramo (Prevođenje):** Circom prevodi tvoj kod u nešto što se zove R1CS (*Rank-1 Constraint System*) – matematički spisak ograničenja koji kompjuteri (verifikatori) razumeju.
3. **Trusted Setup (Ceremonija):** Kritičan momenat. Kroz složenu matematiku, generišu se javni parametri. U protokolu "Groth16" ovo mora da se radi za **svako novo kolo iz početka**. Zato se sve više koristi protokol "PLONK" koji ima "univerzalni setup".
4. **Računanje Witness-a:** Witness (svedok) je kompletna unutrašnja tabela vrednosti tvog kola. Računa se unutar klijenta (na računaru Prover-a) i **nikada se ne šalje na internet**.
5. **Dokaz (Proof):** Prover, koristeći svoj `witness` i parametre iz Setup-a generiše mali kriptografski dokaz (nekoliko bajtova).
6. **Verifikacija:** Verifier (najčešće neki Smart Contract na Ethereumu) primi dokaz, brzinom svetlosti primeni matematiku i kaže Da ili Ne.

Alati za rad sa Circomom

1. **zkREPL** (<https://zkrepl.dev>) - Pomenuto u slajdovima. Ovo je tvoj najbolji prijatelj. To je "igraonica" u browseru gde pišeš kod na jednoj strani ekrana, a desno ubacuješ probne ulazne podatke, generišiš witness i gledaš da li dokaz prolazi. Za početak, apsolutno nikakve instalacije na tvoj kompjuter ti nisu potrebne.
2. **Circomlib** - Ne pišeš sve ručno. Ne moraš da pišeš kako radi heš funkcija. Ljudi iz zajednice su već napisali kod i šablone za to. Ti samo pozoveš `include "circomlib/circuits/poseidon.circom";` i iskoristiš templejt.

Savet za učenje: Kada budeš gledao primere, uvek se fokusiraj na to da razlučiš šta ide u `===` ograničenje. Ograničenje je jedina brana koja sprečava zlonamernog čoveka da lažira dokaz.

PLONK

PLONK je jedan od najpopularnijih i najefikasnijih savremenih protokola za ZKP (Zero-Knowledge Proofs), tačnije spada u kategoriju zk-SNARK-ova.

Cilj protokola: Prover (dokazivač) želi da dokaže Verifier-u (proverivaču) da zna tajnu vrednost x (svedok / witness) za zadatu jednačinu, a da pritom ne otkrije samo x .

Primer jednačine:

$$x^3 + x + 5 = y$$

Javni input (y): Vrednost y je poznata svima. Na primer, ako je zadato $y = 35$, dokazuje se poznavanje tajnog rešenja $x = 3$.

Aritmetizacija izraza

Da bi se izraz proverio, on se mora razbiti na pojedinačne osnovne operacije – množenja i sabiranja (tzv. kapije ili gate-ovi). Uvode se pomoćne promenljive (u i v) koje služe kao međurezultati.

Za jednačinu $x^3 + x + 5 = y$, koraci (kapije) su:

1. $u = x \cdot x$ (računanje x^2)
2. $v = u \cdot x$ (računanje x^3)
3. $v + x + 5 - y = 0$ (provera krajnje jednakosti)

Konkretno vrednosti za primer ($x = 3, y = 35$): $u = 9$ i $v = 27$.

Tabele u PLONK-u

A. Svedok (Witness) tabela

Svaki red u tabeli predstavlja jedan gate (kapiju). Kolone a i b su ulazne vrednosti, dok je kolona c izlazna vrednost iz te kapije. Tabela se popunjava nakon što se uvrste privatni i javni inputi.

Opšta struktura (sa ograničenjima / constraints):

| Kolona a | Kolona b | Kolona c | Ograničenje (Constraint) | Opis

|
| --- | --- | --- | --- |
| x | x | u | $a \cdot b - c = 0$ | Računanje x^2 |
| u | x | v | $a \cdot b - c = 0$ | Računanje x^3 |
| v | x | 0 | $a + b + 5 - y = 0$ | Provera izraza sa javnim inputom y (zato je $c = 0$) |
|

Konkretan izgled tabele za primer ($x = 3, u = 9, v = 27$):

a	b	c
3	3	9
9	3	27
27	3	0

B. Selektor tabela

Sva ograničenja (*constraints*) u PLONK-u moraju se svesti na jedinstvenu, univerzalnu jednačinu u opštem obliku:

$$q_L \cdot a + q_R \cdot b + q_M \cdot a \cdot b + q_O \cdot c + q_C + PI = 0$$

Gde su q vrednosti **selektori** koji "uključuju" ili "isključuju" delove jednačine, a PI predstavlja javni input (*Public Input*).

Tabela selektora za naš primer:

q_L	q_R	q_M	q_O	q_C	PI	Odgovarajuće ograničenje
0	0	1	-1	0	0	$a \cdot b - c = 0$
0	0	1	-1	0	0	$a \cdot b - c = 0$
1	1	0	0	5	$-y$	$a + b + 5 - y = 0$

Ograničenja kopiranja (*Copy Constraints*)

Problem: Ograničenja kapija (*gate constraints*) proveravaju ispravnost svakog reda samo lokalno. Bez dodatnih uslova, redovi ne moraju da dele iste vrednosti (tabela može biti lokalno tačna, a globalni račun pogrešan).

Rešenje: Koriste se *copy constraints* kako bi se osiguralo da su iste varijable kroz različite kapije zaista jednake.

Primer iz kola:

- $a_1 = b_1 = b_2 = b_3 = x$
- $c_1 = a_2 = u$

- $c_2 = a_3 = v$
- Za proveru ovih uslova konstruiše se poseban **polinom** $Z(X)$ (detalji konstrukcije se nalaze u originalnom PLONK radu).

Prelazak sa tabela na polinome

PLONK ne proverava tabele direktno, već proverava da li se odgovarajuće polinomske relacije poklapaju u unapred dogovorenim tačkama (h_1, h_2, h_3) koje predstavljaju redove tabele.

Interpolacija polinoma

Pomoću matematičke interpolacije, kolone iz tabela se pretvaraju u polinome stepena najviše 2 (jer imamo 3 tačke):

- **Svedok (Witness) polinomi:** $A(X), B(X), C(X)$. Računaju se **tek u fazi dokazivanja** za konkretne inpute.
- **Selektorski polinomi:** $q_l(X), q_r(X), q_m(X), q_o(X), q_c(X)$ i $PI(X)$. Računaju se unapred **u fazi kompilacije** (izuzev $PI(X)$).

Primer mapiranja tačaka: $A(h_1) = 3, A(h_2) = 9, A(h_3) = 27$
 $PI(h_1) = 0, PI(h_2) = 0, PI(h_3) = -35$

Konstrukcija ukupnog polinoma $G(X)$

Sve relacije iz tabela se spajaju u jedan zajednički polinom $G(X)$.

(Napomena: Radi jednostavnosti, ovde je izostavljen polinom copy constraint-a $Z(X)$, koji bi se takođe nalazio u krajnjem polinomu).

$$G(X) = q_l(X) \cdot A(X) + q_r(X) \cdot B(X) + q_m(X) \cdot A(X) \cdot B(X) + q_o(X) \cdot C(X) + q_c(X) + PI(X)$$

Deljivost polinoma

- Ako su sva ograničenja ispunjena, polinom $G(X)$ mora biti jednak nuli u tačkama h_1, h_2, h_3 .
- Definise se pomoćni polinom nula: $Z_h(X) = (X - h_1)(X - h_2)(X - h_3)$, za koji važi da je u tim tačkama takođe nula.
- Pošto $G(X)$ ima nule na istim mestima i većeg je stepena, mora postojati količnik-polinom $T(X)$ tako da važi:

$$G(X) = Z_h(X) \cdot T(X)$$

Protokol dokazivanja (Otvaranje polinoma)

Krajnja verifikacija se vrši na sledeći način:

1. **Izazov (Challenge):** *Verifier* bira nasumičnu tačku z .
2. **Dokaz (Commitment):** *Prover* koristi **KZG commitment šemu** da dokaže da zna polinome $A(X), B(X), C(X), T(X)$ i $Z(X)$ tako što šalje samo njihove kriptografske "obaveze" (svežnjeve), a ne cele polinome.
3. **Provera (Verification):** *Verifier* proverava njihova otvaranja u toj konkretnoj nasumičnoj tački z i računa da li važi jednakost:

$$G(z) = Z_h(z) \cdot T(z)$$

Ako relacija važi u nasumično izabranoj tački z , prema matematičkim zakonitostima (Schwartz-Zippel lema), *verifier* dobija stoprocentno uverenje da su sva ograničenja u celom kolu zadovoljena.

Semaphore i MACI

Dobar sistem za glasanje istovremeno treba da obezbedi:

- **pravo glasa** - samo registrovani biraci mogu da glasaju
- **privatnost** - niko ne zna kako je konkretna osoba glasala
- **proverljivost** - svi mogu da provere da li je brojanje glasova bilo posteno
- **otpornost na kupovinu glasova** - birac ne može lako da dokaze za koga je glasao

Semaphore

Semaphore je zero-knowledge protocol koji omogućava korisniku da bez otkrivanja identiteta dokaze da:

- pripada nekoj grupi (tj. ima pravo glasanja)
- nije već poslao glas

Kako semaphore funkcioniše?

1. Korisnik generise svoj Semaphore identitet koji sadrži sve vrednosti: **identityNullifier** i **identityTrapdoor**
2. Komitment njegovog Semaphore identiteta se dodaje u grupu
3. Od svih članova grupe pravi se Merkle stablo
4. Korisnik šalje glas i ZK dokaze da ima pravo glasa i da nije već iskoristio svoj glas
5. Verifier proverava da li je:
 - korisnik član grupe (tj. da li pripada Merkle stablu)
 - glas već iskoriscen

Nullifier je javna kriptografska oznaka koja sprečava duplo glasanje. Racuna se iz dve vrednosti kao $nullifier = Hash(identityNullifier, externalNullifier)$ gde je:

- **identityNullifier** - tajna vrednost vezana za korisnika
- **externalNullifier** - oznaka konkretnog glasanja (poenta je da za razlicita glasanja isti korisnik dobije razlicite nulifier-e)

Koristeci **Membership proof**, korisnik pokazuje:

- da se njegov identity commitment nalazi u grupi
- da zna tajnu vrednost iz koga je taj komitment nastao

Identity commitment se racuna na sledeci nacin: $identityCommitment = Hash(identityNullifier, identityTrapdoor)$, gde su **identityTrapdoor** i **identityNullifier** tajne vrednosti korisnika. U javno Merkle stablo se dodaje samo identityCommitment.

Semafor ne resava dva velika problema:

- posteno brojanje glasova
- kupovinu glasova

MACI

Prethodno navedene mane semafore donekle resava **MACI protokol**.

Glavna ideja:

- glasovi su sifrovani
- rezultat brojanja može da se proveriti pomoću ZK dokaza
- glasac može da promeni glas
- treba strana ne može pouzdano da zna kako je glasac na kraju glasao (i na taj način se smanjuje mogućnost kupovine glasova)

MACI protokol ima tri učesnika:

1. **Glasac** - Registruje se u sistem i generise svoj MACI identitet, nakon čega šalje sifrovani glas koristeći javni ključ koordinatora
2. **Koordinator** - poseduje privatni ključ za dešifrovanje glasova, obrađuje pristigle glasove i objavljuje rezultat zajedno sa zero-knowledge dokazom o korektnosti obrade
3. **Posmatraci** - proveravaju validnost objavljenog dokaza i time potvrđuju da je rezultat pravilno izračunat, bez otkrivanja pojedinačnih glasova

Kako radi MACI protokol?

1. Glasac generise svoj MACI identitet, odnosno par kriptografskih ključeva (javni i privatni ključ)
2. Javni ključ glasača registruje se u sistemu i dodaje u javno Merkle stablo registrovanih korisnika
3. Glasac formira poruku koja sadrži glas i nonce vrednost (redni broj poruke), nakon čega poruku potpisuje svojim privatnim ključem kako bi dokazao autentičnost poruke
4. Potpisana poruka se zatim šifrira javnim ključem koordinatora, tako da samo koordinator može da pročita njen sadržaj
5. Glasac može poslati više poruka tokom trajanja glasanja i na taj način više puta promeniti svoj glas
6. Koordinator dešifruje pristigle poruke i proverava da li je svaka poruka validno potpisana od strane registrovanog korisnika, nakon čega za svakog korisnika prihvata samo poslednji validan glas
7. Koordinator generise ZK dokaz kojim pokazuje da su svi glasovi korektno obrađeni i da je konačni rezultat dobijen pravilnim izvršavanjem definisanog algoritma glasanja, bez otkrivanja pojedinačnih glasova
8. Posmatraci mogu da verifikuju ZK dokaz koordinatora i time proveriti ispravnost objavljenog rezultata

Glavne slabosti:

- poverenje u koordinatora: Koordinator može da dešifruje poruke i vidi pojedinačne glasove.
- ne rešava svaku prinudu: Ako neko kontroliše glasača do kraja glasanja, promena glasa ne pomaže.
- zavisnost od registracije: Ako je ulazna lista glasača loša, protokol ne može sam da popravi taj problem.

FHE

FHE (Fully Homomorphic Encryption) je postupak koji nam omogućava da računamo sa sifrovanim podacima bez njihovog otkrivanja. Postupak je sledeći:

- Korisnik generise javni i tajni ključ
- Privatne podatke sifruje pomoću javnog ključa
- Sifrovane podatke šalje serveru
- Server računa nad sifrovanim podacima
- Rezultat je i dalje sifrovan
- Korisnik ga desifruje pomoću tajnog ključa

Server ne poseduje tajni ključ, pa ne vidi ni ulazne podatke ni rezultat. U sistemima poput aukcija ili glasanja, tajni ključ ne bi trebalo da ima jedna osoba, već se umesto toga često deli između više članova.

Python ima podršku za biblioteke koje podržavaju FHE, koje razvija privatna kompanija Zama. Njihove biblioteke nam omogućavaju da pisemo običnu Python funkciju, a zatim je kompajliramo u FHE kolo. Koristimo dve biblioteke `concrete-python` i `concrete-ml`.

Razlikujemo tri režim rada:

- `clear` - obična predikcija bez FHE
- `simulate` - simulacija FHE kola bez enkripcije - ne stiti podatke, ali pokazuje kakav rezultat možemo očekivati kada isto kolo pokrenemo u pravom FHE režimu
- `execute` - pravo FHE izvršavanje nad sifrovanim podacima

Pre rada pokrenuti sledeću komandu: `pip install concrete-python concrete-ml pandas numpy scikit-learn torch`

Primer 1

Posmatrajmo slučaj u kom korisnik zna funkciju koja se izvršava, ali želi da se izračunavanje izvrši na strani servera nad sifrovanim podacima. U naprednijem scenariju u kojem korisnik ne zna funkciju unapred, FHE sam po sebi nije dovoljan, pa se obično kombinuje sa pristupima kao što su MPC, trusted execution okruženja i zero-knowledge dokazi kako bi se omogućilo sigurno računanje nad podacima.

Računamo:

$$f(x, y) = 3x + 2y + 1$$

```
from concrete import fhe
from concrete.fhe.compilation.utils import inputset

# funkcija koju server izvršava nad sifrovanim podacima
def private_score(x, y):
    return 3*x + 2*y + 1

if __name__ == "__main__":
```

```

# server kompajlira funkciju u FHE kolo, inputset je skup ulaznih vrednosti
koji treba kompajleru
compiler = fhe.Compiler(private_score, {"x": "encrypted", "y": "encrypted"})
inputset = [(x,y) for x in range(8) for y in range(8)]

circuit = compiler.compile(inputset)

# korisnik generise kljuceve
circuit.keygen()
x_private = 4
y_private = 6

# korisnik sifruje podatke
encrypted_input = circuit.encrypt(x_private, y_private)

# server izvrsava funkciju nad sifrovanim podacima
encrypted_result = circuit.run(encrypted_input)

# korisnik desifruje sta je dobio od servera
result = circuit.decrypt(encrypted_result)

print(result)
print("Actual result:", 3 * x_private + 2 * y_private + 1)

```

Primer 2 - Anonimna aukcija

Imamo tri ucesnika - Ana, Bojan i Marko. Svako ima svoju ponudu, ali ne zelimo da otkrijemo same ponude. Zelimo samo da saznamo ko je pobedio. Radi jednostavnosti, pretpostavljamo da su ponude razlicite. Ako postoje jednake ponude, u realnom sistemu bi morala da se definisu dodatna pravila.

```

import numpy as np
from concrete import fhe

def auction_winner(ana, bojan, marko):
    highest_bid = max(ana, bojan, marko)
    return 0 if ana == highest_bid else 1 if bojan == highest_bid else 2

if __name__ == "__main__":
    compiler = fhe.Compiler(auction_winner, {"ana": "encrypted", "bojan":
"encrypted", "marko": "encrypted"})

    bid_values = range(0, 11)

    auction_inputset = [
        (ana, bojan, marko) for ana in bid_values for bojan in bid_values for
marko in bid_values
    ]

```

```

auction_circuit = compiler.compile(auction_inputset)

auction_circuit.keygen()

ana_bid = 7
bojan_bid = 9
marko_bid = 5

encrypted_input = auction_circuit.encrypt(ana_bid, bojan_bid, marko_bid)
encrypted_winner_code = auction_circuit.run(encrypted_input)

winner_code = auction_circuit.decrypt(encrypted_winner_code)

winner_names = {
    0: "Ana",
    1: "Bojan",
    2: "Marko"
}

print("Pobednik:", winner_names[int(winner_code)])

```

Primer 3 - Osetljivi podaci iz baze

Ucitavamo fajl `insurance.csv`, cilj je da predvidimo charges, odnosno medicinski trosak osiguranika. Ulazi mogu biti osetljivi podaci: godine, BMI, broj dece, region, ...

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
from concrete.ml.sklearn import LinearRegression as FHELinearRegression
from time import perf_counter

def load_and_preprocess_data():
    df = pd.read_csv("insurance.csv")
    target_col = "charges"

    X_raw = df.drop(columns=[target_col])
    y_dollars = df[target_col].to_numpy()
    y_thousands = y_dollars / 1000.0

    X = pd.get_dummies(X_raw, drop_first=True)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y_thousands, test_size=0.2, random_state=42)

```

```

x_scaler = StandardScaler()
X_train_scaled = x_scaler.fit_transform(X_train)
X_test_scaled = x_scaler.transform(X_test)

X_all_scaled = x_scaler.transform(X)
y_all = y_thousands

y_train = y_train
y_test = y_test

return X_train_scaled, y_train, X_test_scaled, y_test, X_all_scaled, y_all

if __name__ == "__main__":
    X_train, y_train, X_test, y_test, X_all_scaled, y_all =
load_and_preprocess_data()

    fhe_linear_model = FHELinearRegression(n_bits=12)

    fhe_linear_model.fit(X_train, y_train)
    X_calibration = X_train[:200]

    # kompajliramo model u FHE kolo
    fhe_linear_model.compile(X_calibration)

    from time import perf_counter

    # Poređenje clear, simulate i execute režima na celoj bazi

    # Clear predikcija
    start = perf_counter()
    linear_clear_all = fhe_linear_model.predict(X_all_scaled)
    linear_clear_time = perf_counter() - start

    # FHE simulacija
    start = perf_counter()
    linear_sim_all = fhe_linear_model.predict(X_all_scaled, fhe="simulate")
    linear_sim_time = perf_counter() - start

    # FHE execute
    start = perf_counter()
    linear_execute_all = fhe_linear_model.predict(X_all_scaled, fhe="execute")
    linear_execute_time = perf_counter() - start

    linear_summary = pd.DataFrame({

```

```

"mode": ["clear", "simulate", "execute"],
"MAE": [
    mean_absolute_error(y_all, linear_clear_all),
    mean_absolute_error(y_all, linear_sim_all),
    mean_absolute_error(y_all, linear_execute_all)],
"time_seconds": [
    linear_clear_time,
    linear_sim_time,
    linear_execute_time]])

linear_comparison = pd.DataFrame({
    "actual_dollars": np.ravel(y_all),
    "clear_prediction": np.ravel(linear_clear_all),
    "simulate_prediction": np.ravel(linear_sim_all),
    "execute_prediction": np.ravel(linear_execute_all)
})

print(linear_summary)
print(linear_comparison.head(10))

```

Za linearnu regresiju sada poredimo sva tri režima na svim instancama iz baze.

Dobili smo iste rezultate u clear, simulate i execute režimu jer je model samo linearna funkcija i posle kvantizacije se ista računanja rade na isti način i u običnom i u FHE okruženju. Razlike se obično pojavljuju tek kod složenijih modela, npr. kada postoje nelinearne funkcije, ili kada se zbog ograničene preciznosti i kvantizacije mora raditi aproksimacija računanja, pa FHE izvršavanje više ne može da bude potpuno identično običnom izračunavanju.